Torneo de Komikan Concurrente

Informe Tarea 2 – Sistemas Operativos

Integrantes: Valentina Mella y Paulette Saavedra

1. Diseño del esquema de sincronización

El programa utiliza tres tipos de hilos para organizar el torneo: los clientes, los jueces y las piezas. Cada uno cumple una función dentro de la ejecución de las partidas.

Hilos cliente (N): cada hilo representa un jugador. Se emparejan de dos en dos utilizando `emparejar_mutex` y `emparejar_cond`. El primer jugador que llega espera a un oponente y el segundo completa la pareja, asignándose los roles de Puma y Perros, respectivamente.

Hilo Juez: el segundo jugador crea el hilo Juez de la partida. Este espera una mesa disponible mediante el semáforo `mesas_libres`, inicializado en M, por lo que nunca se supera la cantidad máxima de mesas simultáneas. Luego inicializa el tablero, crea los hilos de las piezas y supervisa el desarrollo de la partida.

Hilos pieza: cada partida cuenta con 13 hilos: uno para el Puma y doce para los Perros. Estos esperan un tiempo aleatorio entre los valores configurados y luego intentan realizar un movimiento.

1.1 Exclusión mutua

Cada partida tiene un mutex propio (`p->mutex`), que protege el tablero, las posiciones de las piezas, las capturas, los movimientos y las condiciones de término.

Cuando una pieza intenta moverse, toma el mutex, revisa las posiciones disponibles y actualiza el tablero dentro de la misma sección crítica. De esta forma, comprobar si una casilla está libre y ocuparla se realiza como una sola operación protegida.

Si dos o más piezas intentan ocupar la misma casilla, solamente una puede realizar primero el movimiento. Las demás encuentran el tablero actualizado y deben calcular nuevamente sus opciones.

Los tiempos de espera se realizan fuera de la sección crítica, evitando mantener bloqueado el tablero mientras las piezas esperan.

1.2 Otros mecanismos utilizados

Además del mutex de cada partida, se utilizan otros mecanismos para coordinar la ejecución:

- **Semáforo:** controla la cantidad de mesas disponibles. Cuando una partida termina, el Juez libera la mesa mediante `sem_post()`.
- **Variables de condición:** permiten avisar a los clientes cuando se completa una pareja y al Juez cuando cambia el estado de la partida.
- **Mutex de impresión:** evita que los mensajes y tableros de distintas partidas se mezclen en consola. Su función es proteger la salida por pantalla, no el estado del tablero.

2. Análisis de riesgos de concurrencia

Deadlock: podría ocurrir si dos hilos esperan recursos que están siendo utilizados entre sí. Para reducir este riesgo, se mantiene un orden definido al adquirir los mutex: primero se toma el mutex de la partida y después el mutex de impresión. Además, el mutex de emparejamiento se libera antes de trabajar con el estado de una partida, y el Juez espera una mesa sin mantener otros recursos bloqueados.

Livelock: se produce cuando los hilos siguen intentando ejecutar operaciones sin conseguir avanzar. En nuestro programa, las piezas no realizan reintentos continuos cuando no encuentran movimientos disponibles, sino que vuelven a esperar un tiempo aleatorio. Esto evita ciclos de reintentos que consuman CPU sin producir avances.

Starvation: los mutex de pthread no garantizan un orden FIFO para acceder a ellos, por lo que una pieza podría perder repetidamente la oportunidad de ejecutar un movimiento. Sin embargo, los retardos aleatorios entre 50 y 300 ms y la corta duración de las secciones críticas reducen la posibilidad de que esto ocurra.

El semáforo tampoco garantiza un orden FIFO entre las parejas que esperan una mesa, aunque cada partida libera el recurso al finalizar.

Abandono y Ctrl+C: el abandono se simula mediante la variable de entorno `FALLAR_JUGADOR`. El Juez revisa periódicamente si alguno de los jugadores corresponde al identificador indicado y, si coincide, termina la partida declarando ganador al oponente.

Por otra parte, el programa contempla la interrupción mediante Ctrl+C. La bandera de detención permite que los hilos reconozcan la solicitud de cierre y finalicen su ejecución. Una vez que terminan, el programa espera la finalización de los hilos y libera los recursos utilizados.

3. Evaluación y justificación del diseño

Se decidió utilizar un único mutex por partida en lugar de implementar un mutex para cada casilla. Aunque la segunda alternativa permitiría mayor paralelismo, también requeriría bloquear varias posiciones para validar los movimientos y las capturas del Puma, aumentando la complejidad de la implementación y el riesgo de deadlock.

Con un solo mutex por tablero, el código resulta más simple y se mantiene la consistencia de los datos compartidos. Además, las secciones críticas son breves y los retardos se realizan fuera de ellas, por lo que la espera entre piezas es reducida.

Cada partida mantiene su propio tablero y mutex, permitiendo que las distintas mesas avancen en paralelo. El semáforo se encarga de controlar la disponibilidad de las mesas sin necesidad de realizar esperas activas.

En cuanto al rendimiento, el uso de un mutex por partida permite mantener tiempos de espera reducidos durante los movimientos, ya que las secciones críticas son cortas. Además, al tener un tablero independiente por mesa, se pueden ejecutar varias partidas simultáneamente, aumentando la concurrencia del torneo.

Se optó por utilizar hilos en lugar de procesos porque comparten memoria y facilitan la comunicación entre los componentes de cada partida, sin necesitar mecanismos adicionales de memoria compartida o IPC.

3.1 Limitaciones

La semilla permite inicializar las decisiones aleatorias de las piezas, pero no garantiza resultados idénticos, ya que el orden de ejecución depende del sistema operativo.

Además, no se establece un límite máximo de movimientos, por lo que algunas partidas podrían prolongarse considerablemente.